

Building Web App Components and Backend Integration

3.1.1 Handling Form Submissions in [Express.js](#)

What does “Working with Forms and APIs” mean?

It refers to **how frontend forms (Angular)** communicate with **backend APIs (Express.js)** to send, receive, and process data.

Why this is Important in Web Applications

Almost every real application uses forms:

- Login / Register
- Contact form
- Profile update
- Checkout form

These forms must:

- Send data to backend
- Get response from backend
- Handle success or errors

High-Level Flow (Very Important ★)

```
css

User fills form (Angular)
↓
Form submitted (HTTP request)
↓
Express API receives data
↓
Validation & processing
↓
Response sent back
```

3.1.1 Handling Form Submissions in Express.js

What is Form Submission?

Form submission is the process of **sending user-entered data from the client to the server** for processing.

In Express.js, form data is usually sent using:

- **POST** (most common)
- **PUT** (for updates)

2. Types of Form Data Sent to Express

Type	Used In
URL-encoded	HTML forms
JSON	Angular / React forms
Multipart	File uploads

Middleware Required to Handle Forms

Express cannot read form data without middleware.

```
app.use(express.json());
```

```
app.use(express.urlencoded({ extended: true }));
```

Why Middleware is Required

- Parses incoming request body
- Makes data available in `req.body`

Handling Form Submission (HTML Form Example)

HTML Form

```
<form method="POST" action="/submit">

  <input type="text" name="username" />

  <button type="submit">Submit</button>

</form>
```

Express Route

```
app.post('/submit', (req, res) => {

  const data = req.body;

  res.send('Form submitted successfully');

});
```

Handling JSON Form Data (Angular Forms)

Modern applications send form data as JSON.

```
app.post('/register', (req, res) => {

  const user = req.body;

  res.status(201).json(user);

});
```

Validating Form Data

Why Validation is Important

- Prevent invalid data
- Improve security
- Ensure data quality

Example Validation

```
if (!req.body.email) {  
    return res.status(400).send('Email is required');  
}
```

Sending Response After Submission

Express can send:

- Success message
- JSON data
- Redirect

JSON Response

```
res.json({ message: 'Form submitted successfully' });
```

Error Handling in Form Submission

```
res.status(400).json({ error: 'Invalid input data' });
```

3.1.2 Sending JSON responses and extracting request data

What is JSON?

JSON (JavaScript Object Notation) is a **lightweight data-interchange format** used to exchange data between **client and server**.

Why JSON is Used in Web APIs

- Easy to read and write
- Language independent
- Native support in JavaScript
- Standard format for REST APIs

Sending JSON Responses in Express.js

Sending a JSON response means **returning structured data from the server to the client** using the HTTP response object.

Using **res.json()**

```
res.json({ message: 'Data received successfully' });
```

With Status Code

```
res.status(200).json({  
  success: true,  
  data: user  
});
```

Why `res.json()` is Preferred

- Automatically sets **Content-Type: application/json**
- Converts JavaScript objects to JSON
- Clean and standard response format

3. Common HTTP Status Codes (Exam ★)

Code	Meaning
200	OK
201	Created
400	Bad Request
401	Unauthorized
404	Not Found
500	Server Error

What is Request Data?

Request data is the information sent by the client to the server as part of an HTTP request.

Types of Request Data in [Express.js](#)

A. Request Body (`req.body`)

Purpose

Used to extract **form data** or **JSON data** sent in POST/PUT requests.

Example

```
app.post('/register', (req, res) => {  
  const userData = req.body;  
  res.json(userData);  
});
```

Required Middleware

```
app.use(express.json());  
app.use(express.urlencoded({ extended: true }));
```

B. Route Parameters (**req.params**)

Used to extract **dynamic values from URL paths**.

Example

```
app.get('/users/:id', (req, res) => {  
  const userId = req.params.id;  
  res.json({ id: userId });  
});
```

C. Query Parameters (**req.query**)

Purpose

Used to extract **optional data** sent after **?** in URL.

Example

```
app.get('/users', (req, res) => {  
  const role = req.query.role;  
  res.json({ role });  
});
```

Example: Complete API Handling

```
app.post('/login', (req, res) => {  
  const { email, password } = req.body;  
  
  if (!email || !password) {  
    return res.status(400).json({ error: 'Missing  
fields' });  
  }  
  
  res.status(200).json({ message: 'Login successful'  
});  
});
```

3.1.3 Connecting Angular Forms to Express APIs

What Does “Connecting Angular Forms to Express APIs” Mean?

It refers to the process of **sending form data from an Angular 21 application to an Express.js backend API** and handling the response.

Why This Connection is Important

Real-world applications require:

- User registration
- Login
- Profile update
- Data storage in database

All these require **frontend–backend communication**.

3. Technologies Involved

Layer	Technology
Frontend	Angular 21
Forms	Reactive Forms
Communication	HttpClient
Backend	Express.js
Data Format	JSON

4. High-Level Data Flow (Very Important ★)

pgsql

Angular Form



HttpClient (POST/PUT)



Express API Route



Controller Logic



JSON Response



Angular UI Update

Steps to implement for connecting angular and express app

Step 1

Generate a new angular application using - ng new <app-name> command

Step 2

Generate a service using - ng generate service <service-name> command and mention all the functions that you want to include inside your front end application.

Example -

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({
  providedIn: 'root',
})
export class UserService {

  private baseUrl = 'http://localhost:3000/api'; (your backend host)

  constructor(private http: HttpClient){}

  createUser(user: any){
    return this.http.post(`${this.baseUrl}/create`, user); (the base api will be hit when you call this service in Front End)
  }

  getAllUsers(){
    return this.http.get<any[]>(`${this.baseUrl}/all`);
  }

  updateUser(id: string, user: any) {
    return this.http.put(
      `${this.baseUrl}/update/${id}`,
      user
    );
  }

  deleteUser(id: string){
    return this.http.delete(`${this.baseUrl}/delete/${id}`)
  }
}
```

Step 3

Go to [app.config.ts](#) file and import FormsModule, provideHttpClient for two way data binding and providing the feature of Http requests in your Front Application

```
import { ApplicationConfig, importProvidersFrom, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { FormsModule } from '@angular/forms';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes),
    provideHttpClient(),
    importProvidersFrom(FormsModule)
  ]
};
```

Step 4

Then create your components that you want to include in your FE application with
- ng generate component <component-name> command

For example user-form and user-display

ng generate component user-form

ng generate component user-display

Then go to user-form component and open [user-form.ts](#) file and write the following thing -

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { UserService } from '../user-service';
import { Router } from '@angular/router';

@Component({
  selector: 'app-user-form',
  imports: [FormsModule],
  standalone: true,
  templateUrl: './user-form.html',
  styleUrls: ['./user-form.css'],
})
export class UserForm {

  user = {name:'', email: '', age: ''};

  constructor(private us: UserService, private router: Router){}

  submitForm(){
    this.us.createUser(this.user).subscribe(()=>{
      this.router.navigate(['/display']);
    })
  }
}
```

Then make your HTML(user-form.html) form like this -

```
<h2>Create User</h2>

<form (ngSubmit)="submitForm()">
  <input [(ngModel)]="user.name" name="name" placeholder="Name"><br><br>
  <input [(ngModel)]="user.email" name="email" placeholder="Email"><br><br>
  <input [(ngModel)]="user.age" name="age" placeholder="Age"><br><br>
  <button type="submit">Add User</button>
</form>
```

With the help of ngModel only the form values will bind to the variable created in the type script file of the component.

Then go to [user-display.ts](#) and write the following code

```
import { Component, OnInit, ChangeDetectorRef } from '@angular/core';
import { CommonModule } from '@angular/common';
import { UserService } from '../user-service';

@Component({
  selector: 'app-user-display',
  imports: [CommonModule],
  standalone: true,
  templateUrl: './user-display.html',
  styleUrls: ['./user-display.css'],
})
export class UserDisplay implements OnInit {

  users: any[] = [];

  constructor(private us: UserService, private cdr: ChangeDetectorRef){}

  ngOnInit() {
    this.us.getAllUsers().subscribe((data)=>{
      console.log('USERS FROM API:', data);
      this.users = data;
      this.cdr.detectChanges();
    })
  }

  loadUsers(){

  }

  deleteUser(id: string){
    this.us.deleteUser(id).subscribe(()=>{
      this.loadUsers();
    })
  }
}
```

CommonModule is important to import because without this you won't be able to use common statements in your html file.

Now write the following code in your user-display.html file

```
<h2>All Users</h2>

<table border="1">
  <tr>
    <th>Name</th>
    <th>Email</th>
    <th>Age</th>
    <th>Action</th>
  </tr>

  <tr *ngFor="let user of users">
    <td>{{ user.name }}</td>
    <td>{{ user.email }}</td>
    <td>{{ user.age }}</td>
    <td>
      <button (click)="deleteUser(user._id)">Delete</button>
    </td>
  </tr>
</table>
```

Step 6

Configure your routes in your Angular Application in [app.route.ts](#)

```
import { Routes } from '@angular/router';
import { UserForm } from '../user-form/user-form';
import { UserDisplay } from '../user-display/user-display';

export const routes: Routes = [
  {path:'', component: UserForm},
  {path:'display', component: UserDisplay}
];
```

Step 7

Most imp step.

Mention `<router-outlet></router-outlet>` in your app.html file. If not written then your application will not work and will never load the component on the page.

3.2 Organizing Code with Models and Controllers

Why Code Organization is Important in Backend

In real backend applications:

- Many APIs exist
- Database operations are complex
- Multiple developers work together

Without proper structure:

- Code becomes messy
- Hard to debug
- Difficult to scale

3.2.1 Structuring Backend with Models, Services & Controllers

A. Recommended Backend Folder Structure

```
pgsql

backend/
├── src/
│   ├── routes/
│   │   └── user.routes.js
│   ├── controllers/
│   │   └── user.controller.js
│   ├── services/
│   │   └── user.service.js
│   ├── models/
│   │   └── user.model.js
│   ├── config/
│   │   └── db.js
│   └── app.js
└── package.json
```

B. Role of Each Layer

1 Routes

- Define API endpoints
- Decide which controller handles the request
- No business logic

Example:

```
router.post('/register', userController.register);
```

2 Controllers

Definition

Controllers handle:

- Request data
- Response logic
- Call services

Controller Example

```
exports.register = async (req, res) => {  
  const user = await userService.createUser(req.body);  
  res.status(201).json(user);  
};
```

3 Models

- Define data structure
- Communicate with database
- Use Mongoose schemas

D. Advantages of This Structure

Benefit	Explanation
Scalability	Easy to add features
Maintainability	Clean separation
Reusability	Logic reused
Testing	Unit testing easy
Teamwork	Parallel development

3.2.2 Creating Data Models using MongoDB (Mongoose)

What is MongoDB?

MongoDB is a **NoSQL, document-oriented database** that stores data in **JSON-like documents**

What is Mongoose?

Mongoose is an **ODM (Object Data Modeling) library** for MongoDB in Node.js.

Why Mongoose is Used

- Schema definition
- Data validation
- Easy CRUD operations
- Model-based structure

.

Installing Mongoose

```
npm install mongoose
```

Connecting to MongoDB

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')
  .then(() => console.log('DB Connected'))
  .catch(err => console.error(err));
```

Creating a User Data Model

User Schema Example

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  password: {
    type: String,
    required: true
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});
```

Creating Model

```
module.exports = mongoose.model('User', userSchema);
```

Creating a Product Data Model

```
const productSchema = new mongoose.Schema({
  title: String,
  price: Number,
  stock: Number,
  category: String
});
```

```
module.exports = mongoose.model('Product',
productSchema);
```

Using Models in Controllers

```
const User = require('../models/user.model');

exports.getAllUsers = async () => {
  return await User.find();
};
```

8. Benefits of Using Mongoose Models

Feature	Benefit
Schema	Structured data
Validation	Data safety
Models	Clean database logic
Methods	Easy CRUD

3.2.3 CRUD operations using Express & MongoDB

Step-by-Step Backend Setup

Step 1: Initialize Project

```
npm init -y
```

Step 2: Install Dependencies

```
npm install express mongoose
```

Step 3: Project Structure (IMPORTANT ★)

```
pgsql
```

```
backend/
```

```
├─ models/
```

```
│   └─ user.model.js
```

```
├─ controllers/
```

```
│   └─ user.controller.js
```

```
├─ routes/
```

```
│   └─ user.routes.js
```

```
├─ app.js
```

```
└─ package.json
```

Step 4: Connect MongoDB (server.js)

```
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

mongoose.connect('mongodb://localhost:27017/crudDB')
  .then(() => console.log('MongoDB Connected'))
  .catch(err => console.error(err));

app.use('/users', require('./routes/user.routes'));

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Step 5: Create Mongoose Model

models/user.model.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  age: Number
});

module.exports = mongoose.model('User', userSchema);
```

Step 6: Create Controller (CRUD Logic)

controllers/user.controller.js

```
const User = require('../models/user.model');

/* CREATE */
exports.createUser = async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json(user);
};

/* READ ALL */
exports getUsers = async (req, res) => {
  const users = await User.find();
  res.json(users);
};

/* READ ONE */
exports.getUserById = async (req, res) => {
  const user = await User.findById(req.params.id);
  res.json(user);
};

/* UPDATE */
exports.updateUser = async (req, res) => {
  const user = await User.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
  );
};
```

```
    res.json(user);
};

/* DELETE */
exports.deleteUser = async (req, res) => {
    await User.findByIdAndDelete(req.params.id);
    res.json({ message: 'User deleted successfully' });
};
```

Step 7: Create Routes

routes/user.routes.js

```
const express = require('express');
const router = express.Router();
const controller =
require('../controllers/user.controller');

router.post('/', controller.createUser);           //
CREATE
router.get('/', controller.getUsers);              // READ
ALL
router.get('/:id', controller.getUserById);        // READ
ONE
router.put('/:id', controller.updateUser);         //
UPDATE
router.delete('/:id', controller.deleteUser);      //
DELETE

module.exports = router;
```

3.3 Authentication & Security Basics

What is Authentication?

Authentication is the process of **verifying the identity of a user** before allowing access to an application.

Why Authentication is Required

Without authentication:

- Anyone can access protected pages
- User data is unsafe
- No control over users

Benefits

- Protects user data
- Enables personalized experience
- Prevents unauthorized access

4. Common Authentication Methods

Method	Description
Email & Password	Most common
Token-based	JWT, session tokens
OAuth	Google, Facebook login
Firebase Auth	Managed authentication

3.3.1 Introduction to Firebase Authentication (Email & Password)

What is Firebase Authentication?

Definition

Firebase Authentication is a **cloud-based authentication service** provided by Google that helps developers **securely authenticate users** in web and mobile applications.

It supports multiple authentication methods such as:

- Email & Password
- Phone authentication
- Google, Facebook, GitHub login

Advantages of Firebase Authentication

Advantage	Explanation
Security	Passwords encrypted automatically
Easy integration	Minimal setup
Scalability	Handles millions of users
Reliability	Managed by Google
Multiple providers	Email, OAuth, phone

Steps -

Step 1: Go to Firebase Console

1. Open browser
2. Visit 🖱️ <https://console.firebase.google.com>
3. Sign in using your **Google account**

Step 2: Create a New Firebase Project

1. Click “Add project”
2. Enter **Project Name**
(Example: `angular-auth-app`)
3. Click **Continue**

Step 3: Google Analytics (Optional)

- You can **Disable** Google Analytics (for students)
- Click **Continue**
- Click **Create Project**

Step 4: Add an App to Firebase Project

1. Inside Firebase Dashboard
2. Click **Web icon** (</>) → *Add Firebase to your web app*

Step 5: Register the App

1. Enter **App nickname**
(Example: **Angular Web App**)
2. Do NOT check Firebase Hosting now (optional)
3. Click **Register app**

Step 6: Get Firebase Configuration

Firebase will generate a config object like this 

```
const firebaseConfig = {  
  apiKey: "YOUR_API_KEY",  
  authDomain: "your-project.firebaseio.com",  
  projectId: "your-project-id",  
  storageBucket: "your-project.appspot.com",  
  messagingSenderId: "XXXX",  
  appId: "XXXX"  
};
```

Important:

Copy this config (used in Angular)

Step 7: Enable Firebase Authentication

1. In left sidebar → **Authentication**
2. Click **Get Started**
3. Go to **Sign-in method**
4. Enable **Email/Password**
5. Click **Save**

Step 8: (Optional) Create Test Users

1. Authentication → **Users**
2. Click **Add user**
3. Enter email & password

3.3.2 Adding user registration and login to Angular frontend

Check the following link for code -

[Registration and Login with Angular and express and firebase](#)

3.3.3 Securing backend routes with Firebase Admin SDK and JWT

Introduction

Securing backend routes is an essential part of modern web applications. Firebase Admin SDK allows backend servers to verify authentication tokens issued by Firebase Authentication, ensuring that only authenticated users can access protected APIs.

What is Firebase Admin SDK?

Firebase Admin SDK is a server-side library provided by Google that enables backend services to interact securely with Firebase Authentication, Firestore, and other Firebase services.

What is JWT?

JWT (JSON Web Token) is a secure token format used to transmit authenticated user information between client and server. Firebase issues JWT tokens after successful login.

Authentication Flow

User logs in using Firebase Authentication on frontend → Firebase issues JWT token → Token is sent with API request → Backend verifies token using Firebase Admin SDK → Access is granted or denied.

Verifying JWT

Token Firebase Admin SDK verifies the ID token sent by the client. If valid, request proceeds; otherwise, access is denied.

Middleware for Route Protection

Authentication middleware checks the token for every protected route before allowing access to controllers.

STEP 1: Create Backend Project

bash

 Copy code

```
mkdir firebase-secure-backend
cd firebase-secure-backend
npm init -y
```

STEP 2: Install Required Packages

bash

 Copy code

```
npm install express firebase-admin cors dotenv
```

STEP 3: Project Folder Structure (IMPORTANT ★)

arduino

 Copy code

```
backend/
├── config/
│   └── firebase.js
├── middleware/
│   └── auth.middleware.js
├── routes/
│   └── protected.routes.js
├── .env
├── app.js
└── package.json
```

STEP 4: Firebase Service Account Setup

1. Go to **Firebase Console**
2. **Project Settings** → **Service Accounts**
3. Click **Generate new private key**
4. Download JSON file
5. Rename it (example):
 `firebase-service-account.json`
6. Place it inside **config/** folder

STEP 5: Firebase Admin SDK Initialization

config/firebase.js

js

Copy code

```
const admin = require('firebase-admin');
const serviceAccount = require('./firebase-service-account.json');

admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});

module.exports = admin;
```

STEP 6: Authentication Middleware (JWT Verification)

middleware/auth.middleware.js

js

Copy code

```
const admin = require('../config/firebase');

const verifyToken = async (req, res, next) => {
  try {
    // Token should be sent as: Authorization: Bearer <token>
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ message: 'Unauthorized: No token' });
    }

    const token = authHeader.split(' ')[1];

    // Verify Firebase JWT
    const decodedToken = await admin.auth().verifyIdToken(token);

    // Attach user info to request
    req.user = decodedToken;

    next();
  } catch (error) {
    res.status(401).json({ message: 'Unauthorized: Invalid token' });
  }
};

module.exports = verifyToken;
```

STEP 7: Create Protected Routes

 routes/protected.routes.js

js

 Copy code

```
const express = require('express');
const router = express.Router();
const verifyToken = require('../middleware/auth.middleware');

// Protected API
router.get('/dashboard', verifyToken, (req, res) => {
  res.json({
    message: 'Protected data accessed successfully',
    user: req.user
  });
});

module.exports = router;
```

STEP 8: Main Server File

 app.js

js

 Copy code

```
const express = require('express');
const cors = require('cors');

const app = express();

app.use(cors());
app.use(express.json());

// Routes
app.use('/api', require('../routes/protected.routes'));

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

STEP 9: How Frontend Sends Token (CONCEPT ★)

Angular / frontend must send Firebase ID token like this:

```
makefile
```

 Copy code

```
Authorization: Bearer <FIREBASE_ID_TOKEN>
```

STEP 10: Complete Authentication Flow (EXAM DIAGRAM ★)

```
pgsql
```

 Copy code

```
User Login (Firebase Auth)
```

```
↓
```

```
Firebase generates JWT (ID Token)
```

```
↓
```

```
Frontend sends API request + token
```

```
↓
```

```
Express Middleware verifies token
```

```
↓
```

```
Protected Route Access Granted
```